



UTM
UNIVERSITI TEKNOLOGI MALAYSIA

**Faculty of
Computing**

SECP3623

Database Programming

2025/2026 SEMESTER 1

Assignment 1 (10%)

Name	:	Elijah She Yu Sheng
Matric Id	:	A23CS0073
Section	:	2

Name	:	Brendan Chia Yan Fei
Matric Id	:	A23CS0211
Section	:	2

Task 1: Database Creation and Integrity Constraints

1. Create a database named `hostel_mgmt_placeyourname`, example: `hostel_mgmt_shamini`.

□ `hostel_mgmt_brendan` || `hostel_mgmt_elijah`

2. Create the following tables with entity and referential integrity.

```
CREATE TABLE room_types(
    type_id INT PRIMARY KEY AUTO_INCREMENT,
    type_name VARCHAR(50),
    rent DECIMAL(10,2),
    deposit DECIMAL(10,2),
    capacity INT
);
```

type_id	type_name	rent	deposit	capacity
NULL	NULL	NULL	NULL	NULL

```
CREATE TABLE rooms(
    room_id INT PRIMARY KEY AUTO_INCREMENT,
    type_id INT,
    room_no VARCHAR(50),
    floor_no INT,
    is_occupied BOOLEAN DEFAULT FALSE,
    FOREIGN KEY(type_id) REFERENCES room_types(type_id)
);
```

room_id	type_id	room_no	floor_no	is_occupied
NULL	NULL	NULL	NULL	NULL

```
CREATE TABLE students(
    student_id INT PRIMARY KEY AUTO_INCREMENT,
    room_id INT,
    fname VARCHAR(50),
    lname VARCHAR(50),
    status BOOLEAN,
    checkin_date DATE,
    email VARCHAR(50),
    FOREIGN KEY(room_id) REFERENCES rooms(room_id)
);
```

student_id	room_id	fname	lname	status	checkin_date	email
NULL	NULL	NULL	NULL	NULL	NULL	NULL

```
CREATE TABLE maintenance(
    maint_id INT PRIMARY KEY AUTO_INCREMENT,
    room_id INT,
    issue_desc VARCHAR(100),
    severity ENUM('LOW', 'MEDIUM', 'HIGH') NOT NULL,
    status ENUM('OPEN', 'RESOLVED') NOT NULL,
    reported_on DATE,
    resolved_on DATE,
    FOREIGN KEY(room_id) REFERENCES rooms(room_id)
) ENGINE=InnoDB;
```

maint_id	room_id	issue_desc	severity	status	reported_on	resolved_on
NULL	NULL	NULL	NULL	NULL	NULL	NULL

```

CREATE TABLE payments(
    payment_id INT PRIMARY KEY AUTO_INCREMENT,
    student_id INT,
    amount DOUBLE,
    paid_on DATE,
    method ENUM('CASH', 'FPX', 'CARD', 'TNG'),
    note VARCHAR(100),
    FOREIGN KEY(student_id)
        REFERENCES students(student_id)
) ENGINE=InnoDB;

```

payment_id	student_id	amount	paid_on	method	note
NULL	NULL	NULL	NULL	NULL	NULL

3. Apply constraints: NOT NULL, PRIMARY KEY, FOREIGN KEY, UNIQUE.

<pre> ALTER TABLE room_types MODIFY COLUMN rent INT NOT NULL; ALTER TABLE room_types MODIFY COLUMN deposit INT NOT NULL; ALTER TABLE room_types MODIFY COLUMN capacity INT NOT NULL; ALTER TABLE rooms MODIFY COLUMN room_no VARCHAR(50) NOT NULL; ALTER TABLE rooms MODIFY floor_no INT; </pre>	<pre> ALTER TABLE room_types MODIFY type_name VARCHAR(50) NOT NULL; ALTER TABLE students MODIFY fname VARCHAR(50) NOT NULL; ALTER TABLE students MODIFY lname VARCHAR(50) NOT NULL; ALTER TABLE students MODIFY status BOOLEAN NOT NULL; ALTER TABLE students MODIFY email VARCHAR(50) NOT NULL; </pre>
<pre> ALTER TABLE maintenance MODIFY issue_desc VARCHAR(100) NOT NULL; ALTER TABLE maintenance MODIFY issue_desc VARCHAR(100) NOT NULL; ALTER TABLE payments MODIFY amount DECIMAL(10,2) NOT NULL; </pre>	✓ 24 15:11:30 ALTER TABLE room_types MODIFY COLUMN rent INT NOT NULL ✓ 25 15:11:30 ALTER TABLE room_types MODIFY COLUMN rent INT NOT NULL ✓ 26 15:11:32 ALTER TABLE room_types MODIFY COLUMN deposit INT NOT NULL ✓ 27 15:11:33 ALTER TABLE room_types MODIFY COLUMN capacity INT NOT NULL ✓ 28 15:11:36 ALTER TABLE rooms MODIFY COLUMN room_no VARCHAR(50) NOT NULL ✓ 29 15:11:37 ALTER TABLE rooms MODIFY floor_no INT ✓ 30 15:11:40 ALTER TABLE room_types MODIFY type_name VARCHAR(50) NOT NULL ✓ 31 15:11:42 ALTER TABLE students MODIFY fname VARCHAR(50) NOT NULL ✓ 32 15:11:43 ALTER TABLE students MODIFY lname VARCHAR(50) NOT NULL ✓ 33 15:11:45 ALTER TABLE students MODIFY status BOOLEAN NOT NULL ✓ 34 15:11:47 ALTER TABLE students MODIFY email VARCHAR(50) NOT NULL ✓ 35 15:11:49 ALTER TABLE maintenance MODIFY issue_desc VARCHAR(100) NOT NULL ✓ 36 15:11:50 ALTER TABLE maintenance MODIFY issue_desc VARCHAR(100) NOT NULL ✓ 37 15:11:52 ALTER TABLE payments MODIFY amount DECIMAL(10,2) NOT NULL

4. Create and insert order: room_types → rooms → students → maintenance → payments. A room is 'occupied' if it has ≥ 1 student with status='ACTIVE'; otherwise 'VACANT'. No orphan FKs.

```
INSERT INTO room_types (type_name, rent, deposit, capacity)
VALUES ('Single', 500.00, 300.00, 2);
```

type_id	type_name	rent	deposit	capacity
1	Single	500	300	2
NULL	NULL	NULL	NULL	NULL

```
INSERT INTO rooms (type_id, room_no, floor_no)
VALUES (1, 'A101', 1);
```

room_id	type_id	room_no	floor_no	is_occupied
2	1	A101	1	0
NULL	NULL	NULL	NULL	NULL

```
INSERT INTO students (room_id, fname, lname, status, checkin_date, email)
VALUES (2, 'Ahmad', 'Zaki', TRUE, '2025-10-26', 'ahmad_z@graduate.utm.my');
```

student_id	room_id	fname	lname	status	checkin_date	email
6	2	Ahmad	Zaki	1	2025-10-26	ahmad_z@graduate.utm.my
NULL	NULL	NULL	NULL	NULL	NULL	NULL

```
UPDATE rooms
SET is_occupied = CASE
    WHEN room_id IN (SELECT room_id FROM students WHERE status=TRUE) THEN TRUE
    ELSE FALSE
END;
```

```
INSERT INTO maintenance (room_id, issue_desc, severity, status, reported_on, resolved_on)
VALUES (2, 'Water leakage in bathroom', 'LOW', 'OPEN', '2025-10-26', NULL);
```

maint_id	room_id	issue_desc	severity	status	reported_on	resolved_on
2	2	Water leakage in bathroom	LOW	OPEN	2025-10-26	NULL
NULL	NULL	NULL	NULL	NULL	NULL	NULL

```
INSERT INTO payments (student_id, amount, paid_on, method, note)
VALUES (6, 500.00, '2025-10-26', 'CASH', 'November Rent');
```

payment_id	student_id	amount	paid_on	method	note
3	6	500.00	2025-10-26	CASH	November Rent
NULL	NULL	NULL	NULL	NULL	NULL

5. Demonstrate schema maintenance:

a. ALTER TABLE: add an email column to students and mark it UNIQUE.

```
ALTER TABLE students
ADD CONSTRAINT uq_email UNIQUE(email);
```

✓ 38 15:13:30 ALTER TABLE students ADD CONSTRAINT uq_email UNIQUE(email)

b. DROP TABLE: create a temporary test table and drop it safely.

```
CREATE TABLE test_table(
    test_id INT PRIMARY KEY,
    text VARCHAR(100)
);

DROP TABLE IF EXISTS test_table;
```

✓ 39 15:14:24 CREATE TABLE test_table(test_id INT PRIMARY KEY, text VARCHAR(100))

✓ 40 15:14:27 DROP TABLE IF EXISTS test_table

```
CREATE TABLE test_table(
    test_id INT PRIMARY KEY,
    text VARCHAR(100)
);

DROP TABLE IF EXISTS test_table;
```

✓ 39 15:14:24 CREATE TABLE test_table(test_id INT PRIMARY KEY, text VARCHAR(100))

✓ 40 15:14:27 DROP TABLE IF EXISTS test_table

Task 2: Data Manipulation and Filtering

1. Data Insertion (DML)

a. room_types

```
INSERT INTO room_types(type_name, rent, deposit, capacity) values
('Double', 800.00, 100.00, 2),
('Premium', 1500.00, 150.00, 1),
('Family', 2500.00, 250.00, 5),
('Single', 500.00, 50.00, 1),
('Double', 1000.00, 100.00, 2),
('Premium', 2000.00, 200.00, 4),
('Family', 2700.00, 300.00, 6),
('Single', 350.00, 50.00, 1),
('Double', 1000.00, 100.00, 2),
('Premium', 3000.00, 300.00, 3);
```

type_id	type_name	rent	deposit	capacity
1	Double	800	100	2
2	Premium	1500	150	1
3	Family	2500	250	5
4	Single	500	50	1
5	Double	1000	100	2
6	Premium	2000	200	4
7	Family	2700	300	6
8	Single	350	50	1
9	Double	1000	100	2
10	Premium	3000	300	3

b. rooms

```
INSERT INTO rooms (room_id, room_no, floor_no, type_id) VALUES
('1', 'A101', 2, 2),
('2', 'A102', 2, 3),
('3', 'A103', 1, 1),
('4', 'B101', 1, 10),
('5', 'B102', 3, 9),
('6', 'B103', 3, 4),
('7', 'C101', 5, 7),
('8', 'C102', 4, 8),
('9', 'C103', 4, 5),
('10', 'D101', 5, 6);
```

room_id	type_id	room_no	floor_no	is_occupied
1	2	A101	2	0
2	3	A102	2	0
3	1	A103	1	0
4	10	B101	1	0
5	9	B102	3	0
6	4	B103	3	0
7	7	C101	5	0
8	8	C102	4	0
9	5	C103	4	0
10	6	D101	5	0

c. students

```
INSERT INTO students(room_id, fname, lname, status, checkin_date, email) values
(1, 'Aaron', 'Chia', 1, '2025-10-01', 'aaronchia@gmail.com'),
(2, 'Brendan', 'Chia', 1, '2025-10-02', 'brendan@gmail.com'),
(3, 'Elijah', 'She', 1, '2025-10-02', 'elijah@gmail.com'),
(4, 'John', 'Doe', 1, '2025-10-03', 'johndoe@gmail.com'),
(NULL, 'Ali', 'Muhammad', 0, NULL, 'ali@gmail.com'),
(5, 'Jerren', 'Tan', 1, '2025-10-03', 'jerren@gmail.com'),
(6, 'Emily', 'Tan', 1, '2025-10-04', 'emily@gmail.com'),
(7, 'Joseph', 'Lee', 1, '2025-10-07', 'joseph@gmail.com'),
(NULL, 'Joseph', 'Lee', 0, NULL, 'joeseph@gmail.com'),
(8, 'Jared', 'Li', 1, '2025-10-08', 'jared@gmail.com'),
(9, 'Charmaine', 'Tang', 1, '2025-10-08', 'charmaine@gmail.com'),
(10, 'Vicky', 'Sim', 1, '2025-10-04', 'vicky@gmail.com');
```

student_id	room_id	fname	lname	status	checkin_date	email
1	1	Aaron	Chia	1	2025-10-01	aaronchia@gmail.com
2	2	Brendan	Chia	1	2025-10-02	brendan@gmail.com
3	3	Elijah	She	1	2025-10-02	elijah@gmail.com
4	4	John	Doe	1	2025-10-03	johndoe@gmail.com
5	NULL	Ali	Muhammad	0	NULL	ali@gmail.com
6	5	Jerren	Tan	1	2025-10-03	jerren@gmail.com
7	6	Emily	Tan	1	2025-10-04	emily@gmail.com
8	7	Joseph	Lee	1	2025-10-07	joseph@gmail.com
9	NULL	Joseph	Lee	0	NULL	joeseph@gmail.com
10	8	Jared	Li	1	2025-10-08	jared@gmail.com
11	9	Charma...	Tang	1	2025-10-08	charmaine@gmail.com
12	10	Vicky	Sim	1	2025-10-04	vicky@gmail.com

d. maintenance

```
INSERT INTO maintenance(room_id, issue_desc, severity, status, reported_on, resolved_on) values
(1, 'Light bulb not working', 'HIGH', 'OPEN', '2025-10-01', NULL),
(2, 'Wall paint in bad condition', 'LOW', 'OPEN', '2025-10-03', NULL),
(3, 'Door knob spoiled', 'HIGH', 'OPEN', '2025-10-03', NULL),
(4, 'Window cannot be securely locked', 'HIGH', 'OPEN', '2025-10-6', NULL),
(5, 'Corridor lights not working', 'LOW', 'RESOLVED', '2025-10-03', '2025-10-05'),
(6, 'No hooks to put on curtains', 'LOW', 'RESOLVED', '2025-10-03', '2025-10-03'),
(7, 'Bed is not stable', 'LOW', 'RESOLVED', '2025-10-03', '2025-10-04'),
(8, 'Cracked floor tiles', 'LOW', 'RESOLVED', '2025-10-03', '2025-10-06'),
(9, 'Toilet pipe cannot be turn off', 'LOW', 'OPEN', '2025-10-03', NULL),
(10, 'Toilet bowl cannot be flushed', 'LOW', 'RESOLVED', '2025-10-07', '2025-10-08');
```

maint_id	room_id	issue_desc	severity	status	reported_on	resolved_on
1	1	Light bulb not working	HIGH	OPEN	2025-10-01	NULL
2	2	Wall paint in bad condition	LOW	OPEN	2025-10-03	NULL
3	3	Door knob spoiled	HIGH	OPEN	2025-10-03	NULL
4	4	Window cannot be securely locked	HIGH	OPEN	2025-10-06	NULL
5	5	Corridor lights not working	LOW	RESOLVED	2025-10-03	2025-10-05
6	6	No hooks to put on curtains	LOW	RESOLVED	2025-10-03	2025-10-03
7	7	Bed is not stable	LOW	RESOLVED	2025-10-03	2025-10-04
8	8	Cracked floor tiles	LOW	RESOLVED	2025-10-03	2025-10-06
9	9	Toilet pipe cannot be turn off	LOW	OPEN	2025-10-03	NULL
10	10	Toilet bowl cannot be flushed	LOW	RESOLVED	2025-10-07	2025-10-08

e. payments

```
INSERT INTO payments(student_id, amount, paid_on, method, note) VALUES
(1, 2000.00, '2025-10-1', 'FPX', 'October Rent'),
(2, 2000.00, '2025-11-2', 'CARD', 'November Rent'),
(3, 2000.00, '2025-12-5', 'TNG', 'December Rent'),
(4, 2000.00, '2025-10-7', 'FPX', 'October Rent'),
(5, 2000.00, '2025-11-9', 'FPX', 'November Rent'),
(6, 1700.00, '2025-10-1', 'CASH', 'October Rent'),
(7, 1700.00, '2025-11-1', 'FPX', 'November Rent'),
(8, 1700.00, '2025-12-1', 'FPX', 'December Rent'),
(9, 1700.00, '2025-11-9', 'FPX', 'November Rent'),
(10, 1700.00, '2025-9-7', 'CARD', 'September Rent');
```


payment_id	student_id	amount	paid_on	method	note
1	1	2000.00	2025-10-01	FPX	October Rent
2	2	2000.00	2025-11-02	CARD	November Rent
3	3	2000.00	2025-12-05	TNG	December Rent
4	4	2000.00	2025-10-07	FPX	October Rent
5	5	2000.00	2025-11-09	FPX	November Rent
6	6	1700.00	2025-10-01	CASH	October Rent
7	7	1700.00	2025-11-01	FPX	November Rent
8	8	1700.00	2025-12-01	FPX	December Rent
9	9	1700.00	2025-11-09	FPX	November Rent
10	10	1700.00	2025-09-07	CARD	September Rent

2. UPDATE and DELETE Operations

a. Update the `is_occupied` column in rooms to `TRUE` for rooms with `ACTIVE` students; otherwise, set it to `FALSE`.

```
UPDATE rooms r
JOIN students s
ON r.room_id = s.room_id
SET r.is_occupied = TRUE
WHERE s.status = TRUE;
```

room_id	type_id	room_no	floor_no	is_occupied
1	2	A101	2	1
2	3	A102	2	1
3	1	A103	1	1
4	10	B101	1	1
5	9	B102	3	1
6	4	B103	3	1
7	7	C101	5	1
8	8	C102	4	1
9	5	C103	4	1
10	6	D101	5	1

b. Delete maintenance records where `status = 'RESOLVED'` and the reported date is older than 60 days (if applicable).

```
DELETE FROM maintenance
WHERE status = 'RESOLVED'
AND reported_on < DATE_SUB(CURDATE(), INTERVAL 60 DAY);
```

✓ 59 15:30:08 DELETE FROM maintenance WHERE status = 'RESOLVED' AND reported_on < DATE_SUB(CURDATE(

3. Data Retrieval and Filtering Queries

a. **BETWEEN:** Display room types where rent is between RM400 and RM800.

```
205 • SELECT *
206 FROM room_types
207 WHERE rent BETWEEN 400 AND 800;
208
```

Result Grid | Filter Rows: | Edit:

	type_id	type_name	rent	deposit	capacity
▶	1	Double	800	100	2
	4	Single	500	50	1
•	NULL	NULL	NULL	NULL	NULL

b. **LIKE:** Retrieve students whose names (fname) start with the letter 'A'.

```
209 • SELECT *
210 FROM students
211 WHERE fname LIKE 'A%';
212
213
```

Result Grid | Filter Rows: | Edit: | Export/Import:

	student_id	room_id	fname	lname	status	checkin_date	email
▶	1	1	Aaron	Chia	1	2025-10-01	aaronchia@gmail.com
	5	NULL	Ali	Muhammad	0	NULL	ali@gmail.com

c. **IN:** Display payments where the payment method is IN ('FPX', 'CARD').

```
213 • SELECT *
214 FROM payments
215 WHERE method IN ('FPX', 'CARD');
216
```

Result Grid | Filter Rows: | Edit: | Export/Import:

	payment_id	student_id	amount	paid_on	method	note
	1	1	2000.00	2025-10-01	FPX	October Rent
	2	2	2000.00	2025-11-02	CARD	November Rent
	4	4	2000.00	2025-10-07	FPX	October Rent
	5	5	2000.00	2025-11-09	FPX	November Rent
	7	7	1700.00	2025-11-01	FPX	November Rent
	8	8	1700.00	2025-12-01	FPX	December Rent
	9	9	1700.00	2025-11-09	FPX	November Rent
	10	10	1700.00	2025-09-07	CARD	September Rent
•	NULL	NULL	NULL	NULL	NULL	NULL

d. Combine AND, OR, and NOT in at least one logical filter query.

<pre>SELECT student_id, fname, lname, status, room_id, checkin_date FROM students WHERE (status = 'ACTIVE' AND checkin_date > '2025-01-01') OR (lname LIKE '%a%') AND NOT (room_id IN (3,5));</pre>					
student_id	fname	lname	status	room_id	checkin_date
1	Aaron	Chia	1	1	2025-10-01
2	Brendan	Chia	1	2	2025-10-02
7	Emily	Tan	1	6	2025-10-04
11	Charmaine	Tang	1	9	2025-10-08

4. Functions and Expressions

a. Aggregate Function: Use at least one among COUNT(), SUM(), AVG(), MIN(), or MAX().

<pre>SELECT COUNT(*) AS total_active_students FROM students WHERE status = TRUE;</pre>	<table> <tr> <td></td><td>total_active_students</td></tr> <tr> <td>▶</td><td>11</td></tr> </table>		total_active_students	▶	11
	total_active_students				
▶	11				
<pre>SELECT ROUND(SUM(amount), 2) AS total_amount_received FROM payments;</pre>	<table> <tr> <td></td><td>total_amount_received</td></tr> <tr> <td>▶</td><td>19000.00</td></tr> </table>		total_amount_received	▶	19000.00
	total_amount_received				
▶	19000.00				
<pre>SELECT ROUND(AVG(rent), 2) AS avg_room_rent FROM room_types;</pre>	<table> <tr> <td></td><td>avg_room_rent</td></tr> <tr> <td>▶</td><td>1440.91</td></tr> </table>		avg_room_rent	▶	1440.91
	avg_room_rent				
▶	1440.91				

b. String Functions: Use any two among UPPER(), LOWER(), LENGTH(), or CONCAT() in your queries.

<pre>SELECT UPPER(CONCAT(fname, ' ', lname)) AS fullname_upper FROM students;</pre>	<pre>SELECT LOWER(email) AS email_lower FROM students;</pre>
---	--

fullname_upper	email_lower
AHMAD ZAKI	aaronchia@gmail.com
AARON CHIA	ahmad_z@graduate.utm.my
BRENDAN CHIA	ali@gmail.com
ELIJAH SHE	brendan@gmail.com
JOHN DOE	charmaine@gmail.com
ALI MUHAMMAD	elijah@gmail.com
JERREN TAN	emily@gmail.com
EMILY TAN	jared@gmail.com
JOSEPH LEE	jerren@gmail.com
JOSEPH LEE	joseph@gmail.com
JARED LI	johndoe@gmail.com
CHARMAINE T...	joseph@gmail.com
VICKY SIM	vicky@gmail.com

Task 3: Data Manipulation and Filtering

1. v_room_status

```

CREATE VIEW v_room_status AS
SELECT r.room_no, rt.type_name, rt.rent, r.floor_no, rt.capacity,
       COUNT(DISTINCT s.student_id) AS n_occupants,
       SUM(CASE WHEN m.status = 'OPEN' THEN 1 ELSE 0 END) AS pending_issues,
       CASE
         WHEN COUNT(DISTINCT s.student_id) > 0 THEN FALSE
         ELSE TRUE
       END AS is_vacant
FROM
  rooms r
  INNER JOIN room_types rt ON r.type_id = rt.type_id
  LEFT JOIN students s ON r.room_id = s.room_id AND s.status = TRUE AND s.room_id IS NOT NULL
  LEFT JOIN maintenance m ON r.room_id = m.room_id
GROUP BY r.room_no, rt.type_name, rt.rent, r.floor_no, rt.capacity;

```

room_no	type_name	rent	floor_no	capacity	n_occupants	pending_issues	is_vacant
A101	Double	800	2	2	1	1	0
A101	Single	500	1	2	2	4	0
A102	Premium	1500	2	1	1	1	0
A103	Single	500	1	2	1	1	0
B101	Double	1000	1	2	1	0	0
B102	Single	350	3	1	1	0	0
B103	Family	2500	3	5	1	0	0
C101	Premium	2000	5	4	1	0	0
C102	Family	2700	4	6	1	1	0
C103	Single	500	4	1	1	0	0
D101	Double	1000	5	2	0	0	1

2. Summary queries

a. Total number of students per room type.

```
SELECT
    rt.type_name AS room_type,
    rt.rent AS rent_price,
    COUNT(s.student_id) AS total_students
FROM room_types rt
LEFT JOIN rooms r ON rt.type_id = r.type_id
LEFT JOIN students s ON r.room_id = s.room_id
GROUP BY rt.type_id, rt.type_name, rt.rent
ORDER BY rt.rent;
```

room_type	rent_price	total_students
Single	350	1
Single	500	1
Double	800	1
Double	1000	1
Double	1000	1
Premium	1500	1
Premium	2000	1
Family	2500	1
Family	2700	1
Premium	3000	1

b. Average rent and total deposit per room type (use ROUND() to 2 d.p.).

```
SELECT
    rt.type_name AS room_type,
    ROUND(SUM(rt.rent), 2) AS total_rent,
    ROUND(AVG(rt.rent), 2) AS average_rent,
    ROUND(SUM(rt.deposit), 2) AS total_deposit
FROM room_types rt
GROUP BY rt.type_name
ORDER BY rt.type_name;
```

	room_type	total_rent	average_rent	total_deposit
►	Double	2800	933.33	300
	Family	5200	2600	550
	Premium	6500	2166.67	650
	Single	850	425	350

c. Monthly payment totals grouped by year & month (ensure your dataset spans ≥ 2 months).

```
SELECT
    YEAR(paid_on) AS pay_year,
    MONTH(paid_on) AS pay_month,
    ROUND(SUM(amount), 2) AS total_payment
FROM payments
GROUP BY YEAR(paid_on), MONTH(paid_on)
ORDER BY pay_year, pay_month;
```

	pay_year	pay_month	total_payment
▶	2025	10	4000
	2025	11	4000
	2025	12	2000

d. Count of OPEN maintenance issues per floor using HAVING COUNT > 2.

```
SELECT
    r.floor_no,
    COUNT(m.maint_id) AS open_issues
FROM maintenance m
JOIN rooms r ON m.room_id = r.room_id
WHERE m.status = 'OPEN'
GROUP BY r.floor_no
HAVING COUNT(m.maint_id) > 2;
```

floor_no	open_issues
----------	-------------

3. Apply SQL functions

ROUND() example:

```
SELECT ROUND(SUM(amount), 2) AS total_amount_received
FROM payments;
```

total_amount_received

18500.00

LOWER() example:

```
SELECT
    LOWER(fname) AS first_name_lower,
    LOWER(lname) AS last_name_lower
FROM students;
```

first_name_lower	last_name_lower
aaron	chia
brendan	chia
elijah	she
john	doe
ali	muhammad
jerren	tan
emily	tan
joseph	lee
joseph	lee
jared	li
charmaine	tang
vicky	sim

CONCAT() example:

```
SELECT
    CONCAT('Room ', r.room_no, ' - ', rt.type_name) AS room_description,
    rt.rent
FROM rooms r
JOIN room_types rt ON r.type_id = rt.type_id
ORDER BY r.room_no;
```

room_description	rent
Room A101 - Premium	1500
Room A102 - Family	2500
Room A103 - Double	800
Room B101 - Premium	3000
Room B102 - Double	1000
Room B103 - Single	500
Room C101 - Family	2700
Room C102 - Single	350
Room C103 - Double	1000
Room D101 - Premium	2000

CASE to categorize room type rent as LOW / MEDIUM / HIGH example:

```
SELECT
    type_name,
    rent,
    CASE
        WHEN rent < 600 THEN 'LOW'
        WHEN rent BETWEEN 600 AND 800 THEN 'MEDIUM'
        ELSE 'HIGH'
    END AS rent_category
FROM room_types
ORDER BY rent;
```

	type_name	rent	rent_category
▶	Single	350	LOW
	Single	500	LOW
	Double	800	MEDIUM
	Double	1000	HIGH
	Double	1000	HIGH
	Premium	1500	HIGH
	Premium	2000	HIGH
	Family	2500	HIGH
	Family	2700	HIGH
	Premium	3000	HIGH